Dockerfile

Qu'est-ce qu'un Dockerfile

- Un Dockerfile est un script contenant des instructions pour construire une image Docker personnalisée.
- Le Dockerfile utilise une syntaxe simple et facile à lire, pouvant être créée et modifiée avec n'importe quel éditeur de texte.
- La structure d'un Dockerfile repose sur un ensemble d'instructions simples, telles que "FROM", "RUN", "COPY", "ENV", "ADD", "EXPOSE", "LABEL", "STOPSIGNAL", "USER", "VOLUME", "WORKDIR", "ONBUILD", etc.
- Une fois qu'une image Docker a été construite, elle peut être facilement déployée dans n'importe quel environnement prenant en charge Docker.
- Vous pouvez facilement intégrer votre Dockerfile dans un pipeline d'intégration continue et de déploiement continu (CI/CD), ce qui vous permet de construire, tester et déployer automatiquement votre application à chaque modification du code.

Qu'est-ce qu'un Dockerfile

- Une fois qu'un Dockerfile est créé, il peut être utilisé pour construire une image en utilisant la commande
  `docker build`
- L'image résultante peut ensuite être exécutée en tant que conteneur en utilisant la commande `docker run` ou `docker exec`.

Qu'est-ce qu'un layer(layers) ?

- Chaque instruction dans un Dockerfile crée une nouvelle couche dans l'image, et l'image finale est composée de toutes les couches empilées les unes sur les autres.
- Chaque instruction ajoute une nouvelle couche à l'image, et chaque couche inclut les instructions spécifiées dans la couche précédente. L'image finale est le résultat de l'ensemble des instructions spécifiées dans le Dockerfile.

```
docker history nom_image
```

```
IMAGE       CREATED         CREATED BY                                      SIZE
abc1234     2 days ago       /bin/sh -c #(nop) CMD ["/bin/bash"]            0B
def5678     2 days ago       /bin/sh -c #(nop) ADD file:2e4c8bbf54a2f9...
```

```
69.3MB
ghj9012     2 days ago       /bin/sh -c #(nop)  MAINTAINER yourname@email.com
0B
```

Dockerfile format

- Voici le format du `Dockerfile` :

```
# Commentaire
INSTRUCTION arguments
```

- L'instruction n'est pas sensible à la casse. Cependant, il est courant de les écrire en MAJUSCULES pour les distinguer plus facilement des arguments.
- Docker exécute les instructions dans un `Dockerfile` dans l'ordre.
- Un fichier `Dockerfile` **doit commencer par une instruction** `FROM`. Cela peut être précédé de directives de parseur, de commentaires et d'arguments globaux définis avec ARGs.
- L'instruction `FROM` spécifie l'image parente à partir de laquelle vous construisez.
- L'instruction `FROM` ne peut être précédée que par une ou plusieurs instructions `ARG`, qui déclarent les arguments utilisés dans les lignes `FROM` du `Dockerfile`.

🏋️ Exercice 1-1 : Création d'une application dockerisée simple nginx "Hello World"

- Dockerfile :

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
#
# docker build -t simple-nginx .
# docker run --rm -it --name test  -p 8080:80 simple-nginx
```

- index.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <meta http-equiv="X-UA-Compatible" content="ie=edge">
   <title>Hello World - Nginx Docker</title>
   <style>
```

```
        h1{
              font-weight:lighter;
              font-family: Arial, Helvetica, sans-serif;
        }
    </style>
</head>
<body>
    <h1>
        Hello World
    </h1>
</body>
</html>
```

- L'option "--rm" indique à Docker de supprimer le conteneur une fois qu'il est arrêté, "-it" est utilisé pour attacher un terminal interactif, et "-p 8080:80" est utilisé pour mapper le port 80 du conteneur sur le port 8080 de notre server.

🏋️ Exercice 1-2 : Création d'une application simple Node.js "Hello World"

-

```
mkdir nodejs-hello
cd nodejs-hello
npm init
npm install express --save
```

- npm créera un fichier package.json qui contient les dépendances de notre application, et le code ajoutera le framework Express en tant que dépendance.
- Créez un fichier index.js contenant un serveur HTTP simple qui fonctionnera sur le port 8080:

```
// Charge le module Express avec la directive `require`
var express = require('express')
var app = express()

// Définit la réponse à la racine de l'URL (/) avec le texte "Hello World!"
app.get('/', function (req, res) {
res.send('Hello World!')
})

// Lance un serveur d'écoute sur le port 8080 et affiche un journal.
app.listen(8080, function () {
```

```
console.log('Application en écoute sur le port 8080 !')
})
```

- Exécutez cette application avec la commande suivante : `node index.js`
- L'application sera accessible à cette adresse IP `http://<adresse_ip_hôte>:8080`. `curl http://localhost:8080/` Cela renverra également "Hello World!" dans le terminal.

🏋️ Exercice 1-2 : Création d'une application dockerisée simple Node.js "Hello World"

- Créez un fichier Dockerfile dans le répertoire racine de la précédente application:

```
   # Utilisation de l'image Node.js Alpine à partir des registres d'images.
   FROM node:8.16.1-alpine

   # Définition du répertoire dans le conteneur à /app pour stocker les
fichiers et lancer notre application.
   WORKDIR /app

   # Copie du fichier package.json dans le répertoire /app avec ses
dépendances.
   COPY package.json /app

   # Installation des dépendances de l'application.
   RUN npm install

   # Copie du reste de l'application dans le répertoire /app.
   COPY . /app

   # Commande pour exécuter notre application, qui est index.js.
   CMD node index.js

   # Exposition du port sur lequel notre application s'exécute, à savoir le
port 3000.
   EXPOSE 3000
   #
   #  docker build -t web .
   # docker run -p 8080:3000 web
```

🏋️ Exercice 1-2 : Création d'une application simple Node.js "Hello World"

- Remarques :

- Nous utilisons l'image officielle Node.js depuis Docker Hub comme point de départ (`FROM node`). Vous pouvez choisir la version de Node.js qui convient à votre application.
- Nous définissons un répertoire de travail dans le conteneur en utilisant l'instruction `WORKDIR` . C'est l'endroit où l'application sera installée et exécutée.
- Nous copions d'abord les fichiers `package.json` dans le répertoire de travail. Cela permettra à Docker de tirer profit de la mise en cache des dépendances lors de la construction de l'image.
- Nous exécutons `npm install` pour installer les dépendances de l'application Node.js.
- Nous copions le reste de l'application dans le répertoire de travail en utilisant l'instruction `COPY` . Assurez-vous que tous les fichiers nécessaires à votre application sont copiés.
- Nous utilisons l'instruction `EXPOSE` pour spécifier le port sur lequel l'application Node.js écoutera. Assurez-vous de remplacer `3000` par le port réel de votre application.
- Enfin, nous utilisons l'instruction `CMD` pour démarrer l'application Node.js. Dans cet exemple, nous utilisons `npm start` , mais vous devrez personnaliser cette commande en fonction de la manière dont votre application Node.js est configurée pour démarrer.

🏋️ Exercice 1-3 : Création d'une application simple python/flask (1)

- Créez un fichier Dockerfile dans un nouveau répertoire:

```
FROM python:3.6
MAINTAINER M. Alain "m.alain@ynov.com"
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
EXPOSE 5000
ENTRYPOINT ["python"]
CMD ["app.py"]
# docker build -t simple-flask-app:latest .
# docker run -d -p 5000:5000 simple-flask-app
# http://<adresse_ip_hôte>:5000
```

- requirements.txt :

```
flask
```

- app.py :

```
from flask import Flask
import os

app = Flask(__name__)

@app.route("/")
def hello():
   return "Flask inside Docker!!"
if __name__ == "__main__":
   port = int(os.environ.get("PORT", 5000))
   app.run(debug=True,host='0.0.0.0',port=port)
```

🏋️ Exercice 1-3 : Création d'une application simple python/flask (2)

- Créez un fichier Dockerfile dans un nouveau répertoire:

```
FROM python:slim-bullseye
LABEL version=v1.0.0
LABEL app=ynov-app
ENV FLASK_APP=app.py
ENV FLASK_ENV=dev
WORKDIR /app
COPY . .
RUN pip3 install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["flask", "run", "--host=0.0.0.0"]
```

- requirements.txt

```
flask
```

- app.py :

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
   return "Hello Ynov !!"
```

Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
    # Utilisation d'une image de base
    FROM ubuntu:20.04

    # Exécute une commande pour afficher "hello world" lorsque le conteneur est
lancé
    CMD echo hello \
    # comment
    world
```

```
    # Utilisation d'une image de base
    FROM ubuntu:20.04

    # Exécute une commande pour afficher "hello world" lorsque le conteneur est
lancé
    CMD echo hello \
    world
```

13/02/20025

Comment

- Docker considère les lignes qui **commencent** par `#` comme des commentaires, sauf si la ligne est une directive de parser directive.
- Un marqueur `#` n'importe où ailleurs dans une ligne est traité comme un argument. Cela permet des déclarations comme :

```
    # Comment
    CMD echo 'we are running some # of cool things'
```

- Les lignes de commentaire sont supprimées avant l'exécution des instructions du Dockerfile, ce qui signifie que le commentaire dans l'exemple suivant n'est pas pris en charge par le shell qui exécute la commande `echo`, et les deux exemples de l'exercice précédent sont équivalents.

Parser directives

- Les directives ne sont pas interprétées par le moteur Docker lors de la création d'une image, mais elles fournissent des informations supplémentaires au parseur pour effectuer des tâches spécifiques.
- Voici quelques exemples de directives de parseur courantes dans un fichier Dockerfile :
  1. `# syntax = <builder>` : Cette directive spécifie la version du générateur (builder) à utiliser pour la construction de l'image. Elle peut être utilisée pour activer des fonctionnalités spécifiques du générateur. Par exemple, `# syntax = docker/dockerfile:1.0` indiquerait d'utiliser la version 1.0 du générateur Dockerfile.
  2. `# escape = <escape-char>` : Cette directive permet de spécifier un caractère d'échappement personnalisé à utiliser dans le Dockerfile. Par défaut, l'échappement est le caractère `\`, mais avec cette directive, vous pouvez le changer pour un autre caractère.
  3. `# syntax = registry/repository:tag` : Cette directive permet de spécifier une image de base personnalisée pour la construction de l'image. Elle est utilisée pour personnaliser la source à partir de laquelle l'image de base est extraite.

Ces directives sont principalement utilisées dans les contextes avancés où vous avez besoin d'un contrôle fin sur la façon dont le fichier Dockerfile est analysé et interprété. Pour la plupart des cas d'utilisation courants, vous n'avez pas besoin d'utiliser ces directives.

Whitespace

Pour des raisons de compatibilité ascendante, les espaces blancs en tête avant les commentaires (`#`) et les instructions (comme `RUN`) sont ignorés, mais déconseillés. Les espaces blancs en tête ne sont pas conservés dans ces cas, et les exemples suivants sont donc équivalents :

```
        # this is a comment-line
    RUN echo hello
RUN echo world
```

```
# this is a comment-line
RUN echo hello
RUN echo world
```

Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Utilisation d'une image de base
FROM ubuntu:20.04
CMD echo "\
      hello\
      world"
```

```
# Utilisation d'une image de base
FROM ubuntu:20.04
# Exécute une commande pour afficher "hello world" lorsque le conteneur est
lancé
CMD echo " \
hello \
      world \
"
```

Whitespace

Notez que les espaces dans les *arguments* des instructions, tels que les commandes suivant `RUN`, sont conservés, de sorte que l'exemple suivant affiche `hello world` avec les espaces en tête tels que spécifiés :

```
# Utilisation d'une image de base
FROM ubuntu:20.04
CMD echo "\
      hello\
      world"
```

```
# Utilisation d'une image de base
FROM ubuntu:20.04
# Exécute une commande pour afficher "hello world" lorsque le conteneur est
lancé
CMD echo " \
hello \
      world \
"
```

Variables d'environnement

```
ENV <key>=<value> ...
```

Nous utilisons les instructions `ENV` pour définir les variables d'environnement. Les variables d'environnement sont notées dans le `Dockerfile` soit avec `$variable_name` soit `${variable_name}`. Elles sont traitées de manière équivalente et la syntaxe entre accolades est généralement utilisée pour résoudre les problèmes liés aux noms de variables sans espaces, comme `${foo}_bar`.

Dans l'exercice précédent, nous utilisons les instructions `ENV` pour définir deux variables d'environnement, `MY_NAME` et `MY_CITY`. Ensuite, dans l'instruction `CMD`, nous utilisons ces variables d'environnement pour personnaliser le message qui sera affiché lorsque le conteneur est lancé.

Vous verrez la sortie du conteneur affichant le message personnalisé en utilisant les variables d'environnement que nous avons définies dans le Dockerfile.

Vous pouvez également personnaliser ces variables d'environnement en utilisant l'option `-e` de `docker run`.

Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Utilisation d'une image de base
FROM ubuntu:20.04

# Définition de variables d'environnement
ENV MY_NAME Robert
ENV MY_CITY Montpellier

# Exécute une commande pour afficher les variables d'environnement
CMD echo "Hello, my name is $MY_NAME and I'm from $MY_CITY"

# docker build -t my-container:v1 .
# docker run my-container:v1
# docker run -e MY_NAME=Alice -e MY_CITY=Paris my-container:v1
```

Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Utilisation d'une image de base
FROM ubuntu:20.04

# Exécute une commande pour afficher les variables d'environnement avec des
valeurs par défaut
CMD echo "Hello, my name is ${MY_NAME:-Guest} and I'm from
${MY_CITY:+NewYork}"
#
# docker build -t my-container:v1 .
# docker run my-container:v1
# docker run -e MY_NAME=Alice -e MY_CITY=Paris my-container:v1
```

Variables d'environnement

La syntaxe `${variable_name}` prend également en charge quelques-uns des modificateurs standards de `bash`, comme spécifié ci-dessous :

- `${variable:-word}` indique que si `variable` est définie, le résultat sera cette valeur. Si `variable` n'est pas définie, le résultat sera `word`.
- `${variable:+word}` indique que si `variable` est définie, le résultat sera `word`, sinon le résultat est une chaîne vide.

Dans tous les cas, `word` peut être une chaîne de caractères, y compris d'autres variables d'environnement.

L'ajout d'un `\` avant la variable permet l'échappement : `\$foo` ou `\${foo}`, par exemple, se traduira par les littéraux `$foo` et `${foo}` respectivement.

🏠

Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Utilisation d'une image de base
FROM ubuntu:20.04

# Définition de variables d'environnement
ENV My_First_Name=Robert
ENV My_First_Name=Olivier MY_NAME=$My_First_Name
ENV My_Last_Name=$MY_NAME

# Exécute une commande pour afficher les variables d'environnement
CMD echo "Hello, my fisr name is $My_First_Name and my last name is
```

```
$My_Last_Name"

# docker build -t my-container:v1 .
# docker run my-container:v1
# docker run -e MY_NAME=Alice -e MY_CITY=Paris my-container:v1
```

Variables d'environnement

La substitution de variables d'environnement utilisera la même valeur pour chaque variable tout au long de l'instruction entière. En d'autres termes, dans cet exemple :

```
# Utilisation d'une image de base
FROM ubuntu:20.04

# Définition de variables d'environnement
ENV My_First_Name=Robert
ENV My_First_Name=Olivier MY_NAME=$My_First_Name
ENV My_Last_Name=$MY_NAME

# Exécute une commande pour afficher les variables d'environnement
CMD echo "Hello, my first name is $My_First_Name and my last name is
$My_Last_Name"
```

- `My_Last_Name` ait une valeur de `Robert`, pas `Olivier`.

FROM

```
FROM [--platform=<platform>] <image> [AS <name>]
```

Or

```
FROM [--platform=<platform>] <image>[:<tag>] [AS <name>]
```

Or

```
FROM [--platform=<platform>] <image>[@<digest>] [AS <name>]
```

🏠

Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
$  docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
aece8493d397: Pull complete
Digest:
sha256:2b7412e6465c3c7fc5bb21d3e6f1917c167358449fecac8176c6e496e5c1f05f
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Permet de récupérer le Digest



Exercice

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Alternative 1: Définition de l'image sans tag ni digest
FROM --platform=linux/amd64 ubuntu AS myimage1
ENV image_var=Valeur1
# Utilisation de la variable image1_var dans CMD
CMD echo "Image 1: $image_var"

# Alternative 2: Définition de l'image avec un tag
FROM --platform=linux/amd64 alpine:3.14 AS myimage2
ENV image_var=Valeur2
# Utilisation de la variable image2_var dans CMD
CMD echo "Image 2: $image_var"

# Alternative 3: Définition de l'image avec un digest
FROM --platform=linux/amd64
ubuntu@sha256:2b7412e6465c3c7fc5bb21d3e6f1917c167358449fecac8176c6e496e5c1f05f
AS myimage3
ENV image_var=Valeur3
# Utilisation de la variable image3_var dans CMD
CMD echo "Image 3: $image_var"

#
# docker build -t image1 .
# docker run image1
# docker run -e image_var=Valeurx image1
```

- L'instruction `FROM` initialise un nouveau stade de construction et définit l'image de base pour les instructions suivantes. En tant que tel, un `Dockerfile` valide doit commencer par une instruction `FROM`. L'image peut être n'importe quelle image valide, et il est particulièrement simple de commencer par **pulling une image** depuis les dépôts publics
- `ARG` est la seule instruction qui peut précéder `FROM` dans le `Dockerfile`.
- `FROM` peut apparaître plusieurs fois dans un seul `Dockerfile` pour créer plusieurs images ou utiliser un stade de construction (one build stage) comme dépendance pour un autre. Notez simplement le dernier ID d'image renvoyé par l'étape précédant chaque nouvelle instruction `FROM`. Chaque instruction `FROM` efface tout état créé par les instructions précédentes.
- Optionnellement, un nom peut être donné à un nouveau stade (new build stage) de construction en ajoutant `AS name` à l'instruction `FROM`. Le" name" peut être utilisé dans les instructions `FROM` et `COPY --from=<name>` ultérieures pour faire référence à l'image construite dans cette étape.
- Les valeurs de `tag` ou `digest` sont facultatives. Si vous en omettez une, le constructeur suppose un étiquetage `latest` par défaut. Le constructeur renvoie une erreur s'il ne peut pas trouver la valeur de `tag`.
- L'indicateur facultatif `--platform` peut être utilisé pour spécifier la plate-forme de l'image si `FROM` fait référence à une image multiplateforme. Par exemple, `linux/amd64`, `linux/arm64`, ou `windows/amd64`.
- Par défaut, la plate-forme cible de la demande de construction est utilisée. Des arguments de construction globaux peuvent être utilisés dans la valeur de cet indicateur, par exemple les arguments de plate-forme automatiques vous permettent de forcer un stade à la plate-forme de construction native (`--platform=$BUILDPLATFORM`) et de l'utiliser pour la compilation croisée vers la plate-forme cible à l'intérieur de l'étape.

ARG

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}

# Définition de la commande d'entrée pour afficher la version d'Ubuntu
CMD ["cat", "/etc/os-release"]

# docker build -t my-ubuntu-image --build-arg UBUNTU_VERSION=18.04 .
```

```
# docker build -t my-ubuntu-image .
# docker run my-ubuntu-image
```

ARG

Les instructions `FROM` prennent en charge les variables qui sont déclarées par les instructions `ARG` qui surviennent avant le premier `FROM`.

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}

# Définition de la commande d'entrée pour afficher la version d'Ubuntu
# CMD ["cat", "/etc/os-release"]

ARG UBUNTU_VERSION=18.04
# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}

# Définition de la commande d'entrée pour afficher la version d'Ubuntu
CMD ["cat", "/etc/os-release"]

# docker build -t my-ubuntu-image --build-arg UBUNTU_VERSION=18.04 .
# docker build -t my-ubuntu-image .```
# docker run my-ubuntu-image```
```

ARG

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}

# Stocker la version d'Ubuntu dans un fichier image_version
RUN echo $UBUNTU_VERSION > /image_version

# Définition de la commande d'entrée pour afficher la version d'Ubuntu depuis
image_version
CMD ["cat", "/image_version"]
```

```
# docker build -t my-ubuntu-image .```
# docker run my-ubuntu-image```
```

ARG

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}
ARG UBUNTU_VERSION
# Stocker la version d'Ubuntu dans un fichier image_version
RUN echo $UBUNTU_VERSION > /image_version

# Définition de la commande d'entrée pour afficher la version d'Ubuntu depuis
image_version
CMD ["cat", "/image_version"]

# docker build -t my-ubuntu-image .```
# docker run my-ubuntu-image```
```

Un `ARG` déclaré avant un `FROM` se trouve en dehors d'une étape de construction, il ne peut donc pas être utilisé dans une instruction après un `FROM`. Pour utiliser la valeur par défaut d'un `ARG` déclaré avant le premier `FROM`, utilisez une instruction `ARG` sans valeur à l'intérieur d'une étape de construction.

ARG est principalement utilisé pour personnaliser la construction de l'image, tandis que ENV est utilisé pour configurer l'environnement d'exécution des conteneurs. Si vous avez besoin de passer des valeurs ou des paramètres lors de la construction de l'image, ARG est approprié. Si vous avez besoin de configurer des variables d'environnement pour les conteneurs, ENV est utilisé.

RUN

🏋️ Exécutez les commandes suivantes dans votre terminal et expliquez les résultats obtenus

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}
```

```
ARG UBUNTU_VERSION

# Stocker la version d'Ubuntu dans un fichier image_version : shell form
RUN echo $UBUNTU_VERSION > /image_version

# Stocker la version d'Ubuntu dans un fichier image_version en utilisant :
exec form
RUN ["/bin/sh", "-c", "echo $UBUNTU_VERSION >> /image_version"]

# Définition de la commande d'entrée pour afficher la version d'Ubuntu depuis
image_version
CMD ["cat", "/image_version"]
# docker build -t my-ubuntu-image .```
# docker run my-ubuntu-image```
```

RUN

La commande `RUN` existe sous deux formes :

- `RUN <command>` (*shell* form, la commande s'exécute dans un shell, qui est `/bin/sh -c` par défaut sous Linux ou `cmd /S /C` sous Windows).
- `RUN ["executable", "param1", "param2"]` (*exec* form)

L'instruction `RUN` exécute des commandes dans une nouvelle couche par-dessus l'image actuelle et committe les résultats. L'image ainsi committée sera utilisée pour l'étape suivante dans le `Dockerfile`.

La superposition d'instructions `RUN` et la génération de commits respectent les concepts fondamentaux de Docker, où les commits sont "des étapes de références" et les conteneurs peuvent être créés à partir de n'importe quel point de l'historique de l'image, à l'instar d'un système de contrôle de source.

La forme *exec* permet d'éviter la manipulation des chaînes de caractères dans le shell et de `RUN` des commandes en utilisant une image de base qui ne contient pas l'exécutable shell spécifié.

Dans la forme *shell*, vous pouvez utiliser un `\` (antislash) pour continuer une instruction `RUN` sur la ligne suivante. Par exemple, considérez ces deux lignes :

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

# Utilisation de l'argument pour définir l'image Ubuntu
FROM ubuntu:${UBUNTU_VERSION}
```

```
# Instructions supplémentaires pour votre Dockerfile
RUN apt-get update && apt-get install -y curl

ARG UBUNTU_VERSION

# Stocker la version d'Ubuntu dans un fichier image_version
RUN echo $UBUNTU_VERSION > /image_version

# Définition de la commande d'entrée pour afficher la version d'Ubuntu depuis
image_version
CMD ["cat", "/image_version"]
# docker build -t my-ubuntu-image .```
# docker run my-ubuntu-image```
```

```
   # Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04
FROM ubuntu:${UBUNTU_VERSION}
# Instructions supplémentaires pour votre Dockerfile
RUN apt-get update && \
    apt-get install -y curl

#
ARG UBUNTU_VERSION
RUN echo $UBUNTU_VERSION > /image_version
CMD ["cat", "/image_version"]
# docker build -t my-ubuntu-image .```
# docker run my-ubuntu-image```
```

CMD

The `CMD` instruction has three forms:

- `CMD ["executable","param1","param2"]` (*exec* form, this is the preferred form)
- `CMD ["param1","param2"]` (as *default parameters to ENTRYPOINT*)
- `CMD command param1 param2` (*shell* form)

Il ne peut y avoir qu'une seule instruction `CMD` dans un fichier Docker. Si vous en répertoriez plusieurs, seule la dernière `CMD` prendra effet.

Le principal objectif d'une instruction `CMD` est de fournir des valeurs par défaut pour un conteneur en cours d'exécution. Ces valeurs par défaut peuvent inclure un exécutable, ou elles peuvent exclure l'exécutable, auquel cas vous devez spécifier également une instruction `ENTRYPOINT`.

Si `CMD` est utilisée pour fournir des arguments par défaut pour l'instruction `ENTRYPOINT`, les deux instructions `CMD` et `ENTRYPOINT` doivent être spécifiées sous la forme d'un tableau JSON.

CMD

Forme exec (recommandée) : `CMD ["executable", "param1", "param2"]`

```
# Dockerfile pour la forme exec (recommandée)
FROM ubuntu:20.04
CMD ["echo", "Hello", "World"]

# docker build -t cmd-exec .
# docker run cmd-exec
```

CMD

Forme default parameters to ENTRYPOINT : `CMD ["param1", "param2"]`

```
# Dockerfile pour la forme default parameters to ENTRYPOINT
FROM ubuntu:20.04
ENTRYPOINT ["echo"]
CMD ["Hello", "World"]
#docker build -t cmd-entrypoint .
# docker run cmd-entrypoint
```

CMD

Forme shell : `CMD command param1 param2`

```
# Dockerfile pour la forme shell
FROM ubuntu:20.04
CMD echo Hello World
# docker build -t cmd-shell .
# docker run cmd-shell
```

CMD vs RUN

Ne confondez pas `RUN` avec `CMD`. `RUN` exécute réellement une commande et consolide le résultat ; `CMD` n'exécute rien lors de la construction, mais spécifie la commande prévue pour l'image.

LABEL

```
# Dockerfile avec LABEL
FROM ubuntu:20.04

# Définition de métadonnées avec l'instruction LABEL
LABEL version="1.0"
LABEL description="This is a sample Docker container with metadata labels."
LABEL maintainer="Your Name <your.email@example.com>"

# Instructions supplémentaires pour votre Dockerfile
CMD echo "Hello, this is a container with metadata labels."
#
# docker build -t labeled-container .
# docker run labeled-container
# docker inspect labeled-container
```

LABEL

```
# Dockerfile avec plusieurs étiquettes dans une seule instruction
FROM ubuntu:20.04

# Définition de métadonnées avec l'instruction LABEL (plusieurs étiquettes sur
une seule instruction)
LABEL version="1.0" \
      description="This is a sample Docker container with metadata labels." \
      maintainer="Your Name <your.email@example.com>"

# Instructions supplémentaires pour votre Dockerfile
CMD echo "Hello, this is a container with metadata labels."
#
# docker build -t labeled-container .
# docker run labeled-container
# docker inspect labeled-container
```

LABEL

```
# Dockerfile avec plusieurs étiquettes sur une seule ligne
FROM ubuntu:20.04

# Définition de plusieurs étiquettes sur une seule ligne
LABEL version="1.0" description="This is a sample Docker container with
metadata labels." maintainer="Your Name <your.email@example.com>"

# Instructions supplémentaires pour votre Dockerfile
CMD echo "Hello, this is a container with metadata labels."
```

```
#
# docker build -t labeled-container .
# docker run labeled-container
# docker inspect labeled-container
```

LABEL

```
    LABEL <key>=<value> <key>=<value> <key>=<value> ...
```

L'instruction `LABEL` ajoute des métadonnées à une image. Un `LABEL` est une paire clé-valeur. Pour inclure des espaces à l'intérieur de la valeur d'un `LABEL`, utilisez des guillemets et des barres obliques inverses comme vous le feriez dans l'analyse en ligne de commande. Voici quelques exemples d'utilisation :

Une fois l'image construite avec les étiquettes, vous pouvez exécuter le conteneur comme d'habitude. Pour afficher les étiquettes d'une image Docker, vous pouvez utiliser la commande `docker inspect`. Cela affichera toutes les métadonnées définies à l'aide de l'instruction LABEL pour cette image.

Une image peut avoir plus d'un label. Vous pouvez spécifier plusieurs labels sur une seule ligne. Vous pouvez toujours choisir de spécifier plusieurs labels dans une seule instruction, de l'une des deux manières suivantes :

EXPOSE

Un exemple d'utilisation de l'instruction EXPOSE dans un Dockerfile pour une application Node.js, accompagné des commandes pour construire et exécuter le conteneur :

```
# Utilisation de l'image Nginx
FROM nginx:latest

# Copie des fichiers du site web dans le conteneur
COPY /path/to/your/website /usr/share/nginx/html

# Définition de l'instruction EXPOSE pour exposer le port 80 (TCP)
EXPOSE 80

# Commande d'entrée pour démarrer le serveur Nginx
CMD ["nginx", "-g", "daemon off;"]
#
# docker build -t nginx-app .
# docker run -p 8080:80 nginx-app
```

EXPOSE

```
EXPOSE <port> [<port>/<protocol>...]
```

L'instruction `EXPOSE` informe Docker que le conteneur écoute sur les ports réseau spécifiés à l'exécution. Vous pouvez spécifier si le port écoute en TCP ou en UDP, et par défaut, c'est en TCP si le protocole n'est pas spécifié.

L'instruction `EXPOSE` ne **publie pas réellement le port**. Elle sert de **documentation entre la personne qui construit l'image et celle qui exécute le conteneur**, indiquant quels ports sont destinés à être publiés. Pour publier effectivement le port lors de l'exécution du conteneur, utilisez le drapeau `-p` avec `docker run` pour publier et mapper un ou plusieurs ports, ou le drapeau `-P` pour publier tous les ports exposés et les mapper vers des ports de haut niveau.

Par défaut, `EXPOSE` suppose le TCP. Vous pouvez également spécifier l'UDP :

```
EXPOSE 80/udp
```

Pour exposer à la fois en TCP et en UDP, incluez deux lignes :

```
EXPOSE 80/tcp
EXPOSE 80/udp
```

Dans ce cas, si vous utilisez `-P` avec `docker run`, le port sera exposé une fois en TCP et une fois en UDP. Gardez à l'esprit que `-P` utilise un port hôte éphémère de haut niveau sur l'hôte, donc le port ne sera pas le même pour TCP et UDP.

Indépendamment des paramètres de `EXPOSE`, vous pouvez les outrepasser au moment de l'exécution en utilisant le drapeau `-p`. Par exemple :

```
$ docker run -p 80:80/tcp -p 80:80/udp ...
```

COPY

```
COPY [--chown=<user>:<group>] [--chmod=<perms>] <src>... <dest>
COPY [--chown=<user>:<group>] [--chmod=<perms>] ["<src>",... "<dest>"]
```

Cette dernière forme est nécessaire pour les chemins contenant des espaces.

L'instruction `COPY` copie de nouveaux fichiers ou répertoires depuis `<src>` et les ajoute au système de fichiers du conteneur sous le chemin `<dest>`.

COPY

Pour ajouter tous les fichiers commençant par "hom" :

```
COPY hom* /mydir/
```

Dans l'exemple ci-dessous, `?` est remplacé par n'importe quel caractère unique, par exemple "home.txt".

```
COPY hom?.txt /mydir/
```

COPY

Le `<dest>` est un chemin absolu ou un chemin relatif à `WORKDIR`, dans lequel la source sera copiée à l'intérieur du conteneur de destination.

L'exemple ci-dessous utilise un chemin relatif et ajoute "test.txt" à `<WORKDIR>/relativeDir/` :

```
COPY test.txt relativeDir/
```

Alors que cet exemple utilise un chemin absolu et ajoute "test.txt" à `/absoluteDir/` :

```
COPY test.txt /absoluteDir/
```

ENTRYPOINT

ENTRYPOINT has two forms:

The *exec* form, which is the preferred form:

```
ENTRYPOINT ["executable", "param1", "param2"]
```

The *shell* form:

```
ENTRYPOINT command param1 param2
```

An `ENTRYPOINT` allows you to configure a container that will run as an executable.

ENTRYPOINT : Alternative 1 - *exec* form :

Dockerfile avec ENTRYPOINT sous forme de *exec* form :

```
# Utilisation de l'image de base
FROM ubuntu:20.04

# Définition de l'instruction ENTRYPOINT sous forme de _exec_ form
ENTRYPOINT ["echo", "Hello", "World"]
# docker build -t entrypoint-exec .
# docker run entrypoint-exec
```

ENTRYPOINT: **Alternative 2 - *shell* form :**

Dockerfile avec ENTRYPOINT sous forme de *shell* form :

```
# Utilisation de l'image de base
FROM ubuntu:20.04

# Définition de l'instruction ENTRYPOINT sous forme de _shell_ form
ENTRYPOINT echo Hello World
# docker build -t entrypoint-exec .
# docker run entrypoint-exec
```

CMD vs ENTRYPOINT

Un exemple clair qui illustre la différence entre CMD et ENTRYPOINT en utilisant deux Dockerfiles distincts, suivis des commandes pour construire et exécuter les conteneurs pour chaque cas.

Exemple avec CMD :

```
# Dockerfile avec l'instruction CMD
FROM ubuntu:20.04

# Définition de l'instruction CMD pour exécuter une commande par défaut
CMD ["echo", "This is the CMD instruction."]
# docker build -t cmd-example .
# docker run cmd-example
# docker run cmd-example echo "Custom command"
```

Exemple avec ENTRYPOINT :

```
# Dockerfile avec l'instruction ENTRYPOINT
FROM ubuntu:20.04

# Définition de l'instruction ENTRYPOINT pour exécuter une commande de base
```

```
ENTRYPOINT ["echo", "This is the ENTRYPOINT instruction."]
# docker build -t cmd-example .
# docker run cmd-example
# docker run cmd-example echo "Custom command"
```

Dockerfile avec CMD et ENTRYPOINT :

```
# Dockerfile avec les instructions CMD et ENTRYPOINT
FROM ubuntu:20.04

# Définition de l'instruction CMD pour une commande par défaut
CMD ["echo", "This is the CMD instruction."]

# Définition de l'instruction ENTRYPOINT pour une commande de base
ENTRYPOINT ["echo", "This is the ENTRYPOINT instruction."]

# docker build -t cmd-entrypoint-example .
# docker run cmd-entrypoint-example
# docker run cmd-entrypoint-example "Custom command"
```

- Lorsque vous exécutez le conteneur sans arguments, l'instruction CMD est utilisée pour exécuter la commande par défaut. Cependant, si vous spécifiez des arguments lors de l'exécution, l'instruction ENTRYPOINT traite ces arguments comme des arguments pour la commande de base. Cela permet à l'utilisateur final de personnaliser l'exécution du conteneur en ajoutant des arguments sans avoir à modifier le Dockerfile.
- La combinaison de CMD et ENTRYPOINT permet de créer une image Docker polyvalente et personnalisable, offrant une commande par défaut qui peut être remplacée par des arguments en fonction des besoins de l'utilisateur final.

CMD vs ENTRYPOINT

Dans l'exemple avec CMD, vous pouvez voir que la commande par défaut est exécutée lors du démarrage du conteneur, mais elle peut être remplacée par une commande personnalisée spécifiée lors de l'exécution.

Dans l'exemple avec ENTRYPOINT, la commande de base est exécutée lors du démarrage du conteneur, et les commandes spécifiées lors de l'exécution sont traitées comme des arguments pour la commande de base.

Cela illustre la différence fondamentale entre CMD (qui peut être facilement remplacée) et ENTRYPOINT (qui définit une commande de base qui peut être complétée par des arguments).

L'instruction CMD et l'instruction ENTRYPOINT sont toutes deux utilisées pour définir la commande qui sera exécutée lorsque le conteneur Docker est démarré. Cependant, il y a une différence importante entre les deux :

Vous pouvez combiner l'instruction CMD avec l'instruction ENTRYPOINT dans un Dockerfile pour créer une image Docker avec une commande par défaut (définie par CMD) qui peut être remplacée par des arguments (spécifiés lors de l'exécution) pour l'instruction ENTRYPOINT. Cette combinaison permet de créer une image flexible, où l'utilisateur final peut personnaliser l'exécution du conteneur en ajoutant des arguments.

VOLUME

```
VOLUME ["/data"]
```

La commande `VOLUME` crée un point de montage avec le nom spécifié et le marque comme contenant des volumes montés à partir de l'hôte natif ou d'autres conteneurs. La valeur peut être un tableau JSON, `VOLUME ["/var/log/"]`, ou une simple chaîne avec plusieurs arguments, telle que `VOLUME /var/log` ou `VOLUME /var/log /var/db`.

La commande `docker run` initialise le volume nouvellement créé avec les données qui existent à l'emplacement spécifié dans l'image de base. Par exemple, considérez l'extrait de Dockerfile suivant :

```
FROM ubuntu
RUN mkdir /myvol
RUN echo "hello world" > /myvol/greeting
VOLUME /myvol
```

Ce Dockerfile résulte en une image qui fait en sorte que la commande `docker run` crée un nouveau point de montage à l'emplacement `/myvol` et copie le fichier `greeting` dans le volume nouvellement créé.

VOLUME

Bien sûr, voici un exemple de Dockerfile et les commandes de build et run pour un conteneur Docker qui utilise l'instruction VOLUME pour créer un volume monté. Dans cet exemple, nous allons créer un volume pour stocker des données.

Dockerfile avec l'instruction VOLUME :

```
# Dockerfile avec l'instruction VOLUME
FROM ubuntu:20.04
```

```
# Crée un répertoire qui sera monté en tant que volume
RUN mkdir /myvol

# Crée un fichier de données dans le répertoire
RUN echo "Ceci est un exemple de données." > /myvol/data.txt

# Définit l'instruction VOLUME pour indiquer le répertoire à monter
VOLUME /myvol

# Commande d'entrée pour afficher le contenu du volume
CMD cat /myvol/data.txt
```

- Construction de l'image avec l'instruction VOLUME :

  ```
  docker build -t volume-example .
  ```

- Exécution du conteneur avec le volume monté :

  ```
  docker run -v my_volume:/app volume-example
  ```

Dans cet exemple, nous créons un répertoire `/app` dans le conteneur et y plaçons un fichier de données `data.txt`. L'instruction VOLUME indique que le répertoire `/app` sera monté en tant que volume. Lors de l'exécution du conteneur, nous utilisons l'option `-v` pour créer un volume nommé `my_volume` et le monter sur le répertoire `/app`.

La commande CMD affiche le contenu du fichier `data.txt` dans le répertoire monté.

Après avoir exécuté le conteneur, le contenu du volume sera accessible à l'extérieur du conteneur, ce qui permet de stocker des données de manière persistante et d'y accéder même lorsque le conteneur n'est pas en cours d'exécution.

```
docker volume ls
```

```
docker volume inspect my_volume
```

USER

```
USER <user>[:<group>]
```

or

```
```

```
USER <UID>[:<GID>]
```

La commande `USER` permet de définir le nom d'utilisateur (ou UID) et éventuellement le groupe d'utilisateur (ou GID) à utiliser comme utilisateur par défaut pour le reste de l'étape actuelle. L'utilisateur spécifié est utilisé pour les instructions `RUN` et, lors de l'exécution, exécute les commandes `ENTRYPOINT` et `CMD` correspondantes.

Il est important de noter que lorsqu'un groupe est spécifié pour l'utilisateur, l'utilisateur n'aura *uniquement* l'appartenance au groupe spécifié. Toute autre appartenance à des groupes configurés sera ignorée.

Si l'utilisateur n'a pas de groupe principal, l'image (ou les instructions suivantes) sera exécutée avec le groupe `root`.

```
# Utilisation d'une image de base légère
FROM alpine:latest

# Définition de l'utilisateur non privilégié
RUN adduser -D myuser

# Changement de l'utilisateur par défaut
USER myuser

# Utilisation de l'instruction "echo" pour créer un fichier
RUN echo "Contenu du fichier créé par myuser" > /home/myuser/sample.txt

# Laissez l'entrée par défaut vide
CMD []
```

```
docker build -t monimage .
```

```
docker run -it monimage /bin/sh
```

```
cd /root
```

```
touch /etc/testfile
```

```
docker run -it --user root myimage1 /bin/sh
```

WORKDIR

```
WORKDIR /path/to/workdir
```

L'instruction `WORKDIR` définit le répertoire de travail pour toutes les instructions `RUN`, `CMD`, `ENTRYPOINT`, `COPY`, et `ADD` qui suivent dans le `Dockerfile`. Si le répertoire spécifié avec `WORKDIR` n'existe pas, il sera créé même s'il n'est pas utilisé dans une instruction ultérieure du `Dockerfile`.

L'instruction `WORKDIR` peut être utilisée plusieurs fois dans un `Dockerfile`. Si un chemin relatif est fourni, il sera relatif au chemin du dernier `WORKDIR` précédemment spécifié. Par exemple :

```
# Définition de l'argument pour la version d'Ubuntu (par défaut à 20.04)
ARG UBUNTU_VERSION=20.04

FROM ubuntu:${UBUNTU_VERSION}

# Écriture du chemin du répertoire actuel dans un fichier
RUN pwd > /image_version

# Définition du répertoire de travail initial
WORKDIR /a

# Écriture du chemin du répertoire actuel dans un fichier
RUN pwd >> /image_version

# Changement du répertoire de travail
WORKDIR b

# Ajout du chemin du répertoire actuel au fichier
RUN pwd >> /image_version

# Changement du répertoire de travail
WORKDIR c

# Ajout du chemin du répertoire actuel au fichier
RUN pwd >> /image_version

# Changement du répertoire de travail
WORKDIR /c

# Ajout du chemin du répertoire actuel au fichier
RUN pwd >> /image_version

# Définition de la commande d'entrée pour afficher la version d'Ubuntu depuis
image_version
```

```
CMD ["cat", "/image_version"]

# docker build -t monimage --build-arg UBUNTU_VERSION=20.04 .
# docker run monimage
```

Si aucun répertoire n'est spécifié, le répertoire de travail par défaut est `/`. En pratique, si vous ne créez pas un Dockerfile à partir de zéro (`FROM scratch`), le `WORKDIR` peut probablement être défini par l'image de base que vous utilisez.

Par conséquent, pour éviter des opérations non voulues dans des répertoires inconnus, il est recommandé de définir explicitement votre `WORKDIR`.
